

Making Retrieval Actually Work

Task 1.5. Chunking, embeddings, hybrid search, rerankers and query transformation.

Retrieval quality is won or lost at the **chunk boundary**. If the passage that answers the question was split in half at ingestion, no query-time technique can put it back together.

Two decisions, two very different costs

BUILD-TIME, BAKED IN

Changing these means a re-index

- Chunking strategy and chunk size
- Embedding model and dimensions
- Which metadata is attached
- Hours or days, plus re-embedding cost

QUERY-TIME, PER REQUEST

Changing these is one parameter

- Semantic or hybrid search
- Metadata filters
- Reranking on or off
- Decomposition, and top-k

THE TIEBREAKER

When retrieval is poor, exhaust the right-hand column first. That is the difference between a config change and a re-ingestion project.

The chunking strategies, and when each wins

STRATEGY	HOW IT SPLITS	BEST FOR
Fixed size	N tokens with an overlap percentage	The robust default. Start here
Semantic	At topic shifts, by embedding similarity	Dense prose with no structure
Hierarchical	Parent and child. Search child, return parent	Structured documents
Custom (Lambda)	Whatever your code decides	Structure only you understand
None	One chunk per file	Pre-chunked or short records

ON THE EXAM

Hierarchical is the underrated one. It searches small chunks for precision and returns the larger parent for context.

Choosing a chunk size

- ◆ **Too large:** each hit drags in irrelevant text, diluting the signal and burning context budget.
- ◆ **Too small:** the matching passage loses the surrounding context needed to answer.
- ◆ A common starting point is around **512 tokens with 20% overlap**, then measure.
- ◆ **Overlap exists to survive bad splits.** It duplicates text so a boundary is less likely to sever an idea.
- ◆ Respect the embedding model's **input limit**. Cohere Embed v3 accepts around 512 tokens; Titan V2 accepts far more.

ON THE EXAM

Chunk size is an empirical question about your corpus, not a universal constant. Anyone who states one correct number has not measured a second corpus.

An engineering firm indexes technical standards documents. Each runs to 200 pages with numbered sections, subsections and defined terms, and answers frequently depend on a clause plus the section heading that scopes it.

With fixed-size chunking at 300 tokens, retrieval returns clauses without their governing section, and the model answers as though a clause applied universally when it applied only to one equipment class. Increasing chunk size to 1,500 tokens fixed the scoping errors but retrieval precision fell sharply and answers became vague.

Which chunking strategy should the firm adopt?

- | | |
|---|---|
| <p>A Semantic chunking, so splits occur at topic boundaries rather than arbitrary token counts</p> | <p>B Hierarchical chunking, searching child chunks and returning the parent chunk to the model</p> |
| <p>C Fixed-size chunking at 800 tokens as a compromise between the two failure modes</p> | <p>D No chunking, indexing each 200-page document as a single chunk</p> |

◆ PAUSE AND DECIDE

Which chunking strategy should the firm adopt?

A Semantic chunking

Splits well on flowing prose. Standards documents are explicitly structured, and semantic splitting ignores the numbered hierarchy that carries the scoping.

C Fixed size at 800 tokens

Splits the difference between two known failures, which usually produces a bit of both. It also ignores document structure entirely.

B Hierarchical: search child, return parent

Precision from small children, scope from the returned parent. It refuses the trade-off rather than splitting the difference.

D One chunk per document

A 200-page chunk exceeds embedding input limits and produces one vector representing everything, which matches nothing precisely.

THE SIGNAL

The scenario walks you through both failure modes on purpose. When small chunks lack context and large chunks lack precision, hierarchical is the strategy that gives you both.

The embedding model must be the **same on both sides**. Documents and queries are compared in one coordinate space, and two models do not share one.

Embedding models on Bedrock

MODEL	DIMENSIONS	REACH FOR IT WHEN
Titan Text Embeddings V2	256, 512, 1024	English-leaning text. The cost-effective default
Cohere Embed Multilingual v3	1024	Genuinely multilingual corpora and queries
Cohere Embed v4	256 to 1536	Text and images together, long inputs
Nova Multimodal Embeddings	256 to 3072	Text, image, video and audio in one space

ON THE EXAM

Multimodal embedding models place different media in the same space, so a text query can retrieve a video segment. That is the capability to recognise, not the dimension counts.

Smaller vectors are cheaper, and almost as good

- ◆ AWS measured Titan V2 at roughly **99% of retrieval accuracy at 512** dimensions versus 1024.
- ◆ At **256** dimensions it retains around **97%**, while quartering vector storage.
- ◆ Fewer dimensions means **less index memory, faster search and lower storage cost**.
- ◆ Embedding is the **cheapest step** in a RAG pipeline. Storage and retrieval are where the money goes.
- ◆ You cannot change dimensions later without re-embedding. Decide before you ingest a billion chunks.

ON THE EXAM

Three per cent accuracy for a quarter of the storage is a real engineering trade, and the exam poses it as one. Large corpora make the smaller vector attractive.

A broadcaster is building search across an archive of recorded programmes, presenter notes, and still photography. Producers want to type a description in English and find the matching moment in a video, the related written note, or a photograph.

The current system embeds transcripts with a text embedding model and stores images separately with keyword tags applied by hand. Producers report that searches find written notes well but rarely surface the right video segment or photograph unless someone tagged it accurately.

What should the broadcaster change?

- ☐ A Use Amazon Nova Multimodal Embeddings so text, video and images occupy one shared vector space
- ☐ B Increase the embedding dimensions from 512 to 1024 to improve retrieval accuracy
- ☐ C Use Amazon Rekognition to auto-generate tags for images and video, then keep keyword search
- ☐ D Add a reranking model to improve the ordering of results across the three content types

What should the broadcaster change?

- A

Nova Multimodal Embeddings, one shared space

The visual content is not in the vector space at all. A multimodal model puts images and video into the same space the text query is embedded into.
- B

512 to 1024 dimensions

Buys a few per cent of accuracy on content that is already searchable. It does nothing for media that was never embedded.
- C

Rekognition auto-tagging plus keyword search

Better tags than humans produce, and still keyword matching. A producer describing a mood or an action will not match a label list.
- D

Add a reranker

Reorders what retrieval returned. If the right video segment was never a candidate, no reranker can promote it.

THE SIGNAL

"Finds written notes well but rarely surfaces video or photographs" localises the failure precisely: text is embedded and media is not. Fix the space, not the ranking.

Everything from here is **query-time**. Search type, filters, reranking, decomposition. One parameter each, changeable per request, with no re-ingestion at all.

Semantic search or hybrid?

SEMANTIC ONLY

Vectors alone

- Excellent on paraphrase and concept
- Blurs part numbers and error codes
- The only option on some vector stores
- Misses rare terms it has never seen

HYBRID

Vectors plus keyword

- Semantic recall, plus exact term matching
- Handles identifiers, names and acronyms
- Usually the better default in production
- Needs a store that supports both

THE TIEBREAKER

S3 Vectors as a knowledge base store supports semantic search only. Managed Knowledge Bases always use hybrid and cannot be set to semantic-only. Your store choice constrains this.

Two-stage retrieval, and what a reranker is for

01

Retrieve wide

Fetch 50 to 100 candidates. Optimise for recall.

02

Rerank

A cross-encoder scores each candidate against the query.

03

Cut

Keep the top 5.
Optimise for precision.

04

Generate

The model sees a short, highly relevant context.

ON THE EXAM

A reranker reads the query and the passage together, which an embedding cannot. Embeddings are computed before anyone asked a question; a reranker is computed after.

Reranking on Bedrock

- ◆ The **Rerank API** scores a set of documents against a query, usable outside Knowledge Bases entirely.
- ◆ Inside a Knowledge Base it is a **setting on Retrieve and RetrieveAndGenerate**. No re-ingestion.
- ◆ **Managed Knowledge Bases rerank by default** with a service-managed model.
- ◆ It adds **latency and cost per query**, so it is a trade rather than free improvement.
- ◆ It cannot promote a chunk that retrieval never returned. Widen top-k first, then rerank.

ON THE EXAM

The last point is the one that appears in questions. Reranking fixes ordering, never coverage. If the answer was not in the candidate set, reranking changes nothing at all.

Explicit or implicit metadata filtering?

EXPLICIT

Your application supplies the filter

- You build the filter object in code
- Use when the constraint is already known
- Tenant, department, document status
- Deterministic and auditable

IMPLICIT

Bedrock infers it from the question

- A model reads the query against your schema
- Use when the constraint is only in the prose
- "Policies updated since March" becomes a filter
- Convenient, and it is an inference

THE TIEBREAKER

Never use implicit filtering for access control. An inferred filter is a best effort, and security controls must be derived from identity, as module 4 scenario 3 showed.

A manufacturer's support assistant retrieves from a knowledge base of 3 million chunks built with fixed-size chunking and Titan Text Embeddings V2. Answers about general procedures are good.

Answers about specific part numbers such as "HX-4471-B" are frequently wrong, returning documentation for similar-looking part numbers. The knowledge base was built six weeks ago and re-ingesting it would take around fourteen hours. Management wants the fastest path to acceptable accuracy.

What should the team try first?

- ☐ A Switch the retrieval configuration to hybrid search so exact part numbers are matched as keywords
- ☐ B Re-ingest using semantic chunking so part numbers stay with their descriptions
- ☐ C Re-embed the corpus with a larger embedding model to better distinguish similar identifiers
- ☐ D Fine-tune a foundation model on the part catalogue so it learns the part numbers

◆ PAUSE AND DECIDE

What should the team try first?

- A

 Switch to hybrid search
Exact identifiers are the known weakness of pure vector search and the exact thing keyword matching fixes. It is a query-time parameter, live in minutes.
- B

 Re-ingest with semantic chunking
Fourteen hours to address a problem that is not about chunk boundaries. Part numbers embed poorly regardless of where the chunk starts.
- C

 Re-embed with a larger model
Bigger embeddings still blur near-identical strings, because that is what embeddings do. Full re-index cost for an unlikely gain.
- D

 Fine-tune on the catalogue
Module 2's rule. This is a retrieval problem, and training does not reliably encode a catalogue of identifiers.

THE SIGNAL

Symptom plus cost, together. Part numbers failing while prose succeeds is the textbook hybrid-search signal, and it happens to be the only option that does not require fourteen hours.

Fixing the question before you search

- ◆ **Decomposition.** A multi-part question becomes several sub-queries, each retrieved separately.
- ◆ **Expansion.** Add synonyms and related terms so a narrow phrasing finds broader matches.
- ◆ **Rewriting.** Turn a conversational follow-up into a standalone query. "What about the other one?" retrieves nothing.
- ◆ In Bedrock, decomposition is set through **orchestrationConfiguration** on *RetrieveAndGenerate*.
- ◆ The plain *Retrieve* API has **no orchestration step**, so decomposition is unavailable there.

ON THE EXAM

Follow-up rewriting is the one people forget. In a multi-turn assistant, most queries are fragments that mean nothing on their own, and a vector store will happily return its nearest neighbours anyway.

Retrieve or RetrieveAndGenerate?

RETRIEVE

Chunks come back to you

- You build the prompt yourself
- Full control of context assembly
- Combine with other data sources
- No orchestration, so no decomposition

RETRIEVEANDGENERATE

A grounded answer comes back

- One call: retrieve, prompt, generate
- Citations returned automatically
- Orchestration enables decomposition
- Less control over the final prompt

THE TIEBREAKER

Managed Knowledge Bases are queried with Retrieve and AgenticRetrieveStream rather than RetrieveAndGenerate. Check which knowledge base type a question describes before choosing.

A financial services firm runs an internal research assistant over a custom knowledge base using the RetrieveAndGenerate API. Single-topic questions are answered well.

Compound questions fail. "Compare our capital requirements under the 2023 framework with the 2026 revision and summarise what changed for retail portfolios" returns material about one framework only, and the comparison is either absent or invented. Retrieval returns five chunks, all concerning the same framework.

What should the firm configure?

- ☐ A Increase top-k from 5 to 25 so chunks about both frameworks are returned
- ☐ B Enable query decomposition through orchestrationConfiguration so each sub-question is retrieved separately
- ☐ C Switch to the Retrieve API and assemble the context in application code
- ☐ D Add a reranking model to surface chunks from both frameworks

◆ PAUSE AND DECIDE

What should the firm configure?

A Raise top-k to 25

Might drag in some 2026 chunks by luck. A single embedded query still points at one region of the space, so it returns more of the same neighbourhood.

C Switch to Retrieve and build context yourself

Moves work into application code and loses the orchestration step, which is precisely the feature that solves this.

B Enable query decomposition

The compound question becomes separate sub-queries, each retrieved on its own terms, so both frameworks are guaranteed coverage rather than hoped for.

D Add a reranker

Reranks the five same-framework chunks into a better order. Coverage again, not ordering.

THE SIGNAL

One embedded query is one point in vector space. A question containing two topics averages into a vector near neither. Decomposition is the fix; more results from one bad vector is not.

Exposing retrieval consistently

- ◆ Wrap retrieval as a **tool the model can call**, rather than always retrieving before every prompt.
- ◆ The model then decides *whether* to search, which avoids retrieving for "thanks, that helps".
- ◆ **MCP** gives you one standard interface, so many agents share one retrieval server.
- ◆ A standardised API in front of several knowledge bases lets you add or move stores without touching callers.
- ◆ Always-on retrieval wastes tokens and injects irrelevant context into conversational turns.

ON THE EXAM

This is where retrieval stops being a pipeline stage and becomes a capability. Module 7 builds on it directly, because a tool the model chooses to call is exactly what an agent is.

Where retrieval went wrong, in order

- ◆ **Is the content in the index at all?** Wrong media type, failed parse, deletion not propagated.
- ◆ **Was the chunk boundary sensible?** Answer split across two chunks, or buried in a huge one.
- ◆ **Did the query embed near it?** Exact identifiers, jargon, or an unrewritten follow-up.
- ◆ **Did a filter exclude it?** Over-restrictive metadata filtering returns a confident nothing.
- ◆ **Was it retrieved but ranked too low?** This, and only this, is what reranking fixes.

THE DIAGNOSTIC ORDER

Work down this list in order. Most wrong answers on retrieval questions come from applying a fix from step five to a problem that lives at step one.

A retailer's assistant retrieves from a knowledge base with hybrid search, reranking enabled, and a metadata filter on department derived from the user's profile. Retrieval quality was good for months.

Since a reorganisation renamed several departments, users in the renamed departments now receive "I could not find information about that" for questions that are clearly covered by indexed documents. Users in unchanged departments are unaffected. An architect has proposed re-ingesting with semantic chunking to improve recall.

What is the actual cause?

- ☐ A Chunk boundaries are splitting the relevant content; re-ingest with semantic chunking
- ☐ B The metadata filter now excludes everything, because profile department names no longer match the values stored on the chunks
- ☐ C The reranker is scoring the relevant chunks too low and should be disabled
- ☐ D Hybrid search is diluting semantic matches with keyword noise; switch to semantic-only

◆ PAUSE AND DECIDE

What is the actual cause?

- A

 Chunk boundaries; re-ingest semantically
Chunking did not change, and it would affect all users equally. Fourteen hours of work aimed at a component that is behaving exactly as before.
- C

 Disable the reranker
A reranker orders what was retrieved. When retrieval returns zero chunks there is nothing to order, and disabling it changes nothing.
- B

 The filter no longer matches the stored values
Only renamed departments are affected, and the rename is the only change. A filter matching nothing returns nothing, which the model reports as not found.
- D

 Switch to semantic-only
Would reduce exact-term matching and affect all users. It does not explain why only renamed departments broke.

THE SIGNAL

The population split is the diagnosis. When one group breaks and another does not, look for what differs between them, not for a general quality problem affecting everyone.

The five things to carry forward

- ◆ **Build-time versus query-time.** Chunking and embeddings mean a re-index. Everything else is one parameter.
- ◆ **Hierarchical chunking** when small chunks lose context and large chunks lose precision.
- ◆ **Same embedding model both sides.** Multimodal embeddings are the only way text retrieves images or video.
- ◆ **Hybrid** for exact identifiers. **Reranking** fixes ordering, never coverage. **Decomposition** for compound questions.
- ◆ Diagnose **in order**: indexed, chunked, embedded, filtered, ranked. Most wrong answers skip to the last step.

ON THE EXAM

Module 6 closes Domain 1 with prompt engineering and prompt governance: templates, versions, approval workflows and regression testing.